

Requirements Implementation Audit

Project Overview .1

The SmartHotel booking system is a comprehensive web application designed to manage hotel bookings, provide intelligent recommendations, calculate trip costs, compare hotels, and .manage loyalty programs

:Technology Stack

- .Frontend: React, Vite, TailwindCSS, React-Router, Leaflet (for maps), Lucide-React (icons) •
- .Backend: Node.js, Express.js, Sequelize ORM •
- .Database: Microsoft SQL Server •
- .External Services: OpenStreetMap (via Overpass API) for Nearby Services •

Implementation Summary .2

Main Implementation	Status	Requirement	ID
Register.jsx , authController.js	IMPLEMENTED	Create a new account	REQ-001
Login.jsx , authController.js	IMPLEMENTED	Login	REQ-002
AuthContext.jsx	IMPLEMENTED	Logout	REQ-003
Login.jsx , authController.js (Recovery mode)	IMPLEMENTED	Password recovery	REQ-004
Profile.jsx , authController.js	IMPLEMENTED	Edit profile	REQ-005
Profile.jsx , authController.js	IMPLEMENTED	Change password	REQ-006
Profile.jsx , authController.js	IMPLEMENTED	Delete account	REQ-007
, adminController.js AdminPortal.jsx	IMPLEMENTED	Add a new hotel	REQ-008
, managerController.js ManagerPortal.jsx	IMPLEMENTED	Edit hotel data	REQ-009
, adminController.js AdminPortal.jsx	IMPLEMENTED	Delete hotel	REQ-010
Hotels.jsx , hotelController.js	IMPLEMENTED	View hotel list	REQ-011
UI for uploading/changing hotel images is missing	NOT IMPLEMENTED	Manage hotel images	REQ-012

Main Implementation	Status	Requirement	ID
DB supports it, UI missing	PARTIALLY IMPLEMENTED	Manage hotel amenities	REQ-013
DB supports it, UI missing	PARTIALLY IMPLEMENTED	Manage provided services	REQ-014
, <code>managerController.js</code> <code>ManagerPortal.jsx</code>	IMPLEMENTED	Add new room	REQ-015
API exists (<code>updateRoom</code>), UI missing	PARTIALLY IMPLEMENTED	Edit room data	REQ-016
, <code>managerController.js</code> <code>ManagerPortal.jsx</code>	IMPLEMENTED	Delete room	REQ-017
, <code>managerController.js</code> <code>ManagerPortal.jsx</code>	IMPLEMENTED	View room list	REQ-018
DB <code>status</code> exists, UI missing	PARTIALLY IMPLEMENTED	Change room status	REQ-019
, <code>managerController.js</code> <code>ManagerPortal.jsx</code>	IMPLEMENTED	Set room price	REQ-020
, <code>managerController.js</code> <code>ManagerPortal.jsx</code>	IMPLEMENTED	Set available rooms count	REQ-021
UI missing	NOT IMPLEMENTED	Manage room images	REQ-022
UI missing	NOT IMPLEMENTED	Manage room specific amenities	REQ-023
<code>Hotels.jsx</code> , <code>hotelController.js</code>	IMPLEMENTED	Search for hotels	REQ-024
Dates/guests filtering missing from main search query	PARTIALLY IMPLEMENTED	Apply search factors	REQ-025

Main Implementation	Status	Requirement	ID
Hotels.jsx , hotelController.js	IMPLEMENTED	Filter search results	REQ-026
, recommendationController.js Hotels.jsx	IMPLEMENTED	Run recommendation algorithm	REQ-027
, recommendationController.js Hotels.jsx	IMPLEMENTED	Calculate match percentage	REQ-028
Hotels.jsx , hotelController.js	IMPLEMENTED	Sort search results	REQ-029
Hotels.jsx	IMPLEMENTED	Show recommended hotel	REQ-030
, recommendationController.js Hotels.jsx	IMPLEMENTED	Show reason for recommendation	REQ-031
Compare.jsx	IMPLEMENTED	Select hotels for comparison	REQ-032
(limit 4) comparisonController.js	IMPLEMENTED	Max limit for compared hotels	REQ-033
Compare.jsx	IMPLEMENTED	Show comparison table	REQ-034
Compare.jsx	IMPLEMENTED	Compare prices, ratings, location, services	REQ-035
Compare.jsx	IMPLEMENTED	Sort comparison table	REQ-036
Compare.jsx	IMPLEMENTED	Filter comparison table	REQ-037
Compare.jsx	IMPLEMENTED	Go directly to hotel from comparison	REQ-038

Main Implementation	Status	Requirement	ID
<code>, comparisonController.js</code> <code>Compare.jsx</code>	IMPLEMENTED	Save previous comparison tables	REQ-039
<code>(html2pdf.js) Compare.jsx</code>	IMPLEMENTED	Export comparison to PDF	REQ-040
<code>, comparisonController.js</code> <code>Compare.jsx</code>	IMPLEMENTED	Handle unavailable services indicator	REQ-041
<code>Compare.jsx</code>	IMPLEMENTED	Remove a hotel from comparison	REQ-042
<code>(Overpass API) HotelDetail.jsx</code>	IMPLEMENTED	Show nearby tourist attractions & services	REQ-043
<code>(Haversine formula) HotelDetail.jsx</code>	IMPLEMENTED	Calculate distance	REQ-044
<code>(react-leaflet) HotelDetail.jsx</code>	IMPLEMENTED	Show map & locations	REQ-045
<code>TripCostCalculator.jsx</code>	IMPLEMENTED	Calculate hotel cost	REQ-046
<code>TripCostCalculator.jsx</code>	IMPLEMENTED	Calculate accommodation cost (days/guests)	REQ-047
<code>TripCostCalculator.jsx</code>	IMPLEMENTED	Calculate transport & food cost	REQ-048
<code>TripCostCalculator.jsx</code>	IMPLEMENTED	Show total trip cost	REQ-049
<code>, favoriteController.js</code> <code>Favorites.jsx</code>	IMPLEMENTED	Add/remove favorites	REQ-050
<code>, favoriteController.js</code> <code>Favorites.jsx</code>	IMPLEMENTED	View favorite hotels	REQ-051

Main Implementation	Status	Requirement	ID
<code>recommendationController.js</code>	IMPLEMENTED	Use favorites for recommendations	REQ-052
Could not locate dynamic weather implementation	UNVERIFIED	City information (time, weather)	REQ-053
<code>, reviewController.js</code> <code>HotelDetail.jsx</code>	IMPLEMENTED	Review management (add, edit, delete, view)	REQ-054
<code>, hotelController.js</code> <code>HotelDetail.jsx</code>	IMPLEMENTED	Calculate average rating	REQ-055
<code>, bookingController.js</code> <code>HotelDetail.jsx</code>	IMPLEMENTED	Create new booking	REQ-056
<code>bookingController.js</code>	IMPLEMENTED	Verify room availability	REQ-057
<code>, bookingController.js</code> <code>MyBookings.jsx</code>	IMPLEMENTED	Confirm/Save/Cancel booking	REQ-058
<code>, managerController.js</code> <code>ManagerPortal.jsx</code>	IMPLEMENTED	Dynamic pricing & Benchmarking	REQ-059
<code>, managerController.js</code> <code>ManagerPortal.jsx</code>	IMPLEMENTED	Flash Deals	REQ-060
<code>managerController.js</code>	IMPLEMENTED	Protect competitor hotels data	REQ-061
<code>, bookingController.js</code> <code>loyaltyController.js</code>	IMPLEMENTED	Loyalty (calculate, add, view points)	REQ-062
<code>bookingController.js</code>	IMPLEMENTED	Redeem points for voucher/discount	REQ-063
<code>useCurrency.js</code>	IMPLEMENTED	Currency Support	REQ-064

Detailed Requirement Implementation .3

REQ-001 / REQ-004 💎 Password Recovery (Security Questions)

Status: IMPLEMENTED **How It Was Implemented:** During registration, the user selects two security questions from a dropdown and provides answers. During password recovery (in the `Login.jsx` interface via "Forgot Password"), the system prompts the user with those exact two questions. The user must answer correctly to reset the password. There are no email verifications.

:Frontend Implementation

`frontend/src/pages/Register.jsx` •

:Backend Implementation `frontend/src/pages/Login.jsx` •

`backend/src/routes/authRoutes.js` •

:API `backend/src/controllers/authController.js` •

`POST /api/auth/register` •

:Important Functions `POST /api/auth/verify-answers` •

`authController.js` in `verifySecurityAnswers()` •

REQ-027 / REQ-028 / REQ-031 💎 Recommendation Engine & Match Percentage

Status: IMPLEMENTED **How It Was Implemented:** The system calculates a deterministic recommendation score out of ~100 based on Star Rating (Weight 1.5), Trip Type Alignment (Business/Family/Couple), Price Proximity (Weight 2.0), Amenity Overlap, Review Rating, and Active Flash Deals. The city is properly excluded from the scoring weights as it is used as a hard filter. The match percentage is displayed as a `% Match` badge. Match reasons (e.g., "Ideal for :Couples") are returned and displayed. **Frontend Implementation**

`frontend/src/pages/Hotels.jsx` •

:Backend Implementation `frontend/src/components/HotelCard.jsx` •

:API `backend/src/controllers/recommendationController.js` •

:Important Functions `GET /api/recommendations` •

- `recommendationController.js` in `getRecommendations()` •
- `.rendering logic HotelCard.jsx` •

REQ-032 to REQ-042 💎 Hotel Comparison

Status: IMPLEMENTED **How It Was Implemented:** Users can add up to 4 hotels to a comparison matrix. The backend aggregates the data into a unified matrix, resolving empty fields with a fallback object (`available: false`) which renders as a "not available" indicator. The UI allows sorting, filtering, and exporting the table to PDF using `html2pdf.js` . The user can also save the `:comparison list` to their profile. **Frontend Implementation**

- :Backend Implementation** `frontend/src/pages/Compare.jsx` •
- :API** `backend/src/controllers/comparisonController.js` •
- `POST /api/comparison/matrix` •
- :Important Functions** `POST /api/comparison/saved` •
- `comparisonController.js` in `getSideBySideComparison()` •
- `Compare.jsx` in `handleExportPDF()` •

REQ-043 to REQ-045 💎 Map & Nearby Services

Status: IMPLEMENTED **How It Was Implemented:** The application uses `react-leaflet` for rendering maps and the Overpass API to fetch real-time OSM data for nearby tourist attractions, restaurants, cafes, pharmacies, ATMs, and hospitals. The Haversine formula is used directly in the frontend to calculate the distance between the hotel and the fetched point of interest. **Frontend :Implementation**

- :Important Functions** `frontend/src/pages/HotelDetail.jsx` •
- `HotelDetail.jsx` in `getDistanceFromLatLonInKm()` •
- `.component hook for Leaflet MapUpdater()` •

REQ-046 to REQ-049 💎 Trip Cost Calculator

Status: IMPLEMENTED **How It Was Implemented:** An interactive calculator multiplies the room price by the number of rooms and nights. It also fetches the `avg_daily_food_cost` and `avg_daily_transport_cost` from the city, and multiplies them by the total number of guests

:(adults + children) and trip days. **Frontend Implementation**

`frontend/src/pages/TripCostCalculator.jsx` •

REQ-061 💎 Protect Competitor Hotels Data

Status: IMPLEMENTED **How It Was Implemented:** In the Manager Portal, the `getCompetitorBenchmarking` endpoint fetches other hotels in the same city. However, it explicitly aggregates the data (calculating `marketAvgPrice`, `marketAvgStarRating`, etc.) and returns only the averages to the frontend, strictly protecting the individual competitor data.

:Backend Implementation

:Important Functions `backend/src/controllers/managerController.js` •

`getCompetitorBenchmarking()` •

REQ-008 & REQ-010 💎 Hotel Manager & Hotel Architecture

Status: IMPLEMENTED **How It Was Implemented:** Following the strict business rule of 1 Hotel = 1 Manager, the Admin creates a Manager account without selecting a hotel. The backend automatically creates a new Hotel inside a SQL transaction, links it to the new manager, and assigns a unique local image path. Deleting the manager account automatically deletes the associated hotel and safely cleans up dependent records (Bookings, Favorites, FlashDeals).

:Backend Implementation

:API `backend/src/controllers/adminController.js` •

`POST /api/admin/users` •

`DELETE /api/admin/users/:id` •

Google Meet Demonstration Guide

Password Recovery .1

Browser: Open `/login`, click "Forgot Password". **Action:** Show that it asks for the two custom security questions defined during registration. **Code:** Open `frontend/src/pages/Login.jsx` `(verifySecurityAnswers)` `backend/src/controllers/authController.js` and

Recommendation Engine .2

Browser: Open `/hotels` , execute a search. **Action:** Highlight the "% Match" and the match reason (e.g., "Ideal for Couples"). **Code:** Open `backend/src/controllers/recommendationController.js` to show that city is omitted from `.scoring weights`, and how `recommendationScore` is calculated

Competitor Benchmarking .3

Browser: Login as Hotel Manager, open Manager Portal. **Action:** Show the "Competitor Benchmarking" tab and the market averages. **Code:** Open `backend/src/controllers/managerController.js` (`getCompetitorBenchmarking`) to prove that competitor arrays are aggregated and never sent back directly, protecting competitor `.data`

Admin Architecture (Atomic Creation/Deletion) .4

Browser: Login as System Admin. **Action:** Click "Create Hotel Manager". Note the ABSENCE of a "Select Hotel" dropdown. Create a manager. **Code:** Open `backend/src/controllers/adminController.js` (`createUser`) to show the SQL Transaction `( sequelize.transaction() )` that atomically creates both the Hotel and the Manager together

Maps & Nearby Services .5

Browser: Open a Hotel Details page. **Action:** Scroll to the Leaflet Map. Click the filters (ATMs, Pharmacies, Restaurants). **Code:** Open `frontend/src/pages/HotelDetail.jsx` to show the `.Overpass API` query and the `getDistanceFromLatLonInKm` Haversine implementation

Missing / Partial Requirements

Room and Hotel Management UI .1

What is missing: The UI for editing room details, changing room status •
(available/booked/out of service), and uploading hotel/room images

What exists: The `Room` and `Hotel` database models fully support these fields (`status` , •
• `updateRoom` `is_available` , `main_image`). The backend API has
• `ManagerPortal.jsx` **To fix:** Add image upload fields and status dropdowns in •

Search Filters by Date and Guests .2

What is missing: The main search endpoint (`getHotels`) does not filter hotels out if they •
• are fully booked for the selected dates

• **What exists:** Filtering by City, Price, Rating, and Amenities •

To fix: Update `hotelController.js` to include a date-overlap availability check on the •
• `Rooms` association

Unverified Requirements

City Information (Weather/Best Time to Visit) .1

Code for fetching dynamic weather or "best time to visit" was not clearly identified in the •
• main UI flows

File Cleanups Performed

Temporary files removed: `verifyDb.js` , `verifyApi.js` , `testDelete.js` , 18 •
• `patch_hotelcard.js` , `checkDocker.sh` , etc

• **Result:** The project root is clean and free of testing/debugging artifacts •